Table des matières

Cours 04 — Bâtir l'étoile des ventes 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Ce qui est une vente · INTERMEDIATE 2
Étape 2 — La règle, `int_ventes` · INTERMEDIATE 3
Étape 3 — La date, `dim_date` · MARTS 5
Étape 4 — Les clés de substitution, `dim_produit` et `dim_membre` · MARTS 6
Étape 5 — Le fait, `fct_ventes` · MARTS 7
Étape 6 — Lire l'étoile, puis la réconcilier · MARTS 8
Étape 7 — Versionner · MARTS 10
À retenir 11

Cours 04 — Bâtir l'étoile des ventes

Cours 4/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
	✓ ici	

Ce que vous allez faire

Au cours 02, vous avez dessiné l'étoile : le fait des ventes et ses trois dimensions. Le cours 03 a bâti les quatre vues du `STAGING`, une par table de la `RAW` dont l'étoile a besoin : `stg_tickets`, `stg_lignes`, `stg_produits` et `stg_membres`. Ce cours bâtit l'étoile à partir de ces vues. D'abord l'`INTERMEDIATE`, la couche qui clarifie : `int_ventes` y écrit, une seule fois, ce qui est une vente. Puis les `MARTS` : les trois dimensions, chacune avec sa clé, et le fait des ventes. Le cours finit par le contrôle qui décide si l'étoile se livre : son montant doit retomber, au cent près, sur celui que la caisse a imprimé.

les vues du `STAGING` → `int_ventes` → `dim_date`, `dim_produit`, `dim_membre`, `fct_ventes` → la réconciliation avec la caisse

Les fichiers à écrire se donnent en pseudocode, et le code complet est dans `Codes_04_AE.md`, à ouvrir après avoir essayé.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, dbt et Git ; la seconde pour le notebook de DuckDB, qui s'ouvre à l'étape 1. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dbt ls
REM Attendu : Found 4 models, 4 sources, puis les quatre vues, de brulerie.staging.stg_lignes à
REM brulerie.staging.stg_tickets, et les quatre sources.
```

Ce cours part du projet tel que le cours 03 l'a laissé. S'il manque une vue, reprenez la fin du cours 03 avant de continuer : les modèles de ce cours lisent les quatre.

Étape 1 — Ce qui est une vente · INTERMEDIATE

La caisse n'encaisse pas tout ce qu'elle imprime. Deux codes le disent : `type_ticket` (`VENTE` ou `REMBOURSEMENT`) et `statut` (`PAYE` ou `ANNULE`). Décider ce qui compte comme une vente est une règle métier, et une règle métier s'écrit une seule fois. Elle va dans l'`INTERMEDIATE` : le `STAGING` rend la source lisible sans rien écarter, et le fait des `MARTS` recevra des ventes déjà triées. Cette étape regarde les tickets ; l'étape 2 écrit la règle.

1.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « AE Cours 04 », puis attacher l'entrepôt dans la première cellule :

```
attach if not exists 'plateforme/entrepot/brulerie.duckdb' as brulerie (read_only);
-- Attendu : brulerie paraît sous Attached databases.
```

1.2 Compter les tickets par type et par statut, dans une cellule neuve. `count(*)` suffit : la vue `stg_tickets` garde un seul exemplaire de chaque ticket.

```
-- 1. les tickets, par type et par statut
select type_ticket, statut, count(*) as tickets
from brulerie.stg_tickets
group by ____, ____
order by type_ticket, statut;
-- Attendu : REMBOURSEMENT PAYE 1, VENTE ANNULE 15, VENTE PAYE 1511.
```

1.3 Regarder les premiers tickets annulés.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- 1. les cinq premiers tickets annulés : ce qu'ils valaient, ce qu'ils ont rapporté
select ticket_id, sous_total, mode_paiement, encaisse
from brulerie.stg_tickets
where statut = '____'
order by ticket_id
limit 5;
-- Attendu : 5 tickets, du 158435 au 158596, chacun avec un sous-total, aucun mode de
-- paiement (NULL) et 0.00 encaissé.
```

Un ticket annulé l'a été avant le paiement : ses lignes et ses montants restent imprimés, mais rien n'a été encaissé. Les 15 du mois portent 136,00 $ de sous-totaux. Ils sortent des ventes.

1.4 Regarder le remboursement.

```sql
-- 1. le remboursement du mois : son ticket, sa date, la vente d'origine, ses montants
select ticket_id, horodatage, ticket_origine, sous_total, encaisse
from brulerie.stg_tickets
where type_ticket = '____';
-- Attendu : 1 ticket, le 159782 du 27 juin, qui rembourse la vente 159676 : -4.00 de
-- sous-total, -4.60 encaissé.
```

Une chocolatine vendue le 25 juin, rendue le 27. Le remboursement est payé : de l'argent est ressorti de la caisse. Il reste dans les ventes, en négatif, parce que les deux tickets disent ce qui s'est passé. Pour La Brûlerie, une vente est donc une ligne d'un ticket payé : les annulés sortent, le remboursement reste.

1.5 Fermer le notebook, pour que dbt puisse écrire dans l'entrepôt : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient. Laisser la seconde fenêtre ouverte : le notebook y revient à l'étape 6.

Étape 2 — La règle, `int_ventes` · INTERMEDIATE

`int_ventes` porte les lignes des tickets payés, chacune avec la date et le membre de son ticket : tout ce que le fait demandera, et rien de ce qu'il écarte. C'est le premier modèle qui en lit d'autres. Un modèle lit un autre modèle par `{{ ref('stg_lignes') }}`, comme il lit une source par `source()` : dbt sait alors que `int_ventes` vient après `stg_lignes` et `stg_tickets`, et les bâtit dans cet ordre.

2.1 Créer les dossiers des deux couches, dans la première fenêtre, puis ouvrir le fichier du projet.

```
mkdir 02_ae\models\intermediate
mkdir 02_ae\models\marts
notepad 02_ae\dbt_project.yml
```


© 2026 iAlexMG. Tous droits réservés.

Chaque couche reçoit sa matérialisation. L'`INTERMEDIATE` reste en vues : seul le projet le lit. Les `MARTS` deviennent des tables : les rapports les liront souvent, et une table se calcule une fois, au `dbt run`, là où une vue rejoue sa requête à chaque lecture. Le fichier change en deux endroits, le commentaire 2 et les deux couches ajoutées sous `staging` :

```
# 02_ae/dbt_project.yml : le projet dbt de La Brûlerie
name: brulerie
version: "1.0.0"
# 1. le profil de connexion à lire dans profiles.yml
profile: brulerie

# 2. un dossier de models/ par couche ; le STAGING et l'INTERMEDIATE en vues, les MARTS en tables
models:
  brulerie:
    staging:
      +materialized: view
    intermediate:
      +materialized: ____
    marts:
      +materialized: ____
```

Enregistrer, puis fermer le Bloc-notes.

2.2 Écrire le modèle.

```
notepad 02_ae\models\intermediate\int_ventes.sql
```

La jointure rattache chaque ligne à son ticket par leur numéro commun. `as l` et `as t` donnent un nom court à chaque table, et `l.ticket_id` dit de laquelle vient la colonne : les deux en ont une. La date de vente se prend en `date`, pas en `timestamp` : elle deviendra la clé vers la dimension des dates, un jour par ligne (étape 3), et un horodatage comme `2025-06-01 08:06:36` n'y retrouverait aucun jour. L'heure de la vente ne se perd pas : `stg_tickets` la garde. Accepter la création, puis écrire :

```
-- 02_ae/models/intermediate/int_ventes.sql : les lignes des tickets payés, avec leur date et leur membre
select
    -- 1. la ligne, avec la date et le membre de son ticket
    l.ticket_id,
    l.ligne,
    t.horodatage::____ as date_vente,
    t.membre_id,
    -- puis, à écrire, de la ligne : produit_id, quantite, prix_unitaire, rabais et montant
-- 2. les lignes et leur ticket, lus par ref()
from {{ ref('____') }} as l
inner join {{ ref('____') }} as t on t.ticket_id = l.ticket_id
```


© 2026 iAlexMG. Tous droits réservés.

```
-- 3. ce qui est une vente : un ticket payé ; les annulés sortent, le remboursement reste
where ____
```

Enregistrer, puis fermer le Bloc-notes.

2.3 Bâtir le modèle seul. `--select` ne bâtit que le modèle nommé.

```
dbt run --select int_ventes
REM Attendu : 1 of 1 OK created sql view model main.int_ventes, puis Done. PASS=1.
```

dbt avertit aussi que `models.brulerie.marts` est une configuration inutilisée : le dossier est encore vide, et l'avertissement disparaît à l'étape 3.

Étape 3 — La date, `dim_date` · MARTS

Une dimension de dates ne vient d'aucune source : on la fabrique, un jour par ligne, sans trou. Les ventes ne connaissent que les jours où le café a vendu ; la dimension connaît tous les jours du mois, et ce qu'on veut savoir de chacun, son nom et s'il tombe en fin de semaine. Sa clé est la date elle-même : une date est unique, stable et lisible, et le fait la porte déjà.

Pour remplir	**L'outil**
un jour par ligne	`generate_series(début, fin, interval 1 day)`, les deux bornes comprises ; `as jours(jour)` nomme la table et sa colonne
le jour de la semaine	`isodow(jour)` : 1 pour lundi, 7 pour dimanche
son nom	`case isodow(jour) when 1 then 'lundi' … end`

`generate_series` s'apprend au Cours 05 de la formation SQL avancé, `Theorie_05_SQL.md`, section 6, et sur des dates au Cours 09, `Theorie_09_SQL.md`, section 2. DuckDB a bien `dayname`, mais il rend les noms en anglais : le nom du jour se décode ici, une seule fois, là où la dimension naît.

3.1 Écrire le modèle.

```
notepad 02_ae\models\marts\dim_date.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/models/marts/dim_date.sql : les 30 jours de juin 2025, un par ligne
select
    -- 1. la clé : la date elle-même
    jour::date as date_jour,
    -- 2. le nom du jour, du lundi (1) au dimanche (7)
    case isodow(jour)
        when 1 then 'lundi' when 2 then 'mardi' when 3 then 'mercredi'
        -- puis, à écrire : jeudi, vendredi, samedi et dimanche
```


© 2026 iAlexMG. Tous droits réservés.

```
    end as nom_jour,
    -- 3. la fin de semaine : le samedi et le dimanche
    isodow(jour) >= ____ as est_fin_de_semaine
-- 4. une ligne par jour, du 1er au 30 juin, les deux bornes comprises
from generate_series(date '____', date '____', interval 1 day) as jours(jour)
```

Enregistrer, puis fermer le Bloc-notes.

3.2 Bâtir la dimension.

```
dbt run --select dim_date
REM Attendu : 1 of 1 OK created sql table model main.dim_date, puis Done. PASS=1.
```

Étape 4 — Les clés de substitution, `dim_produit` et `dim_membre` · MARTS

Le produit et le membre ont déjà un numéro, celui de la caisse et celui de la carte. Chaque dimension reçoit quand même sa propre clé, une clé de substitution : un code que l'entrepôt fabrique lui-même à partir du numéro. Le numéro reste dans la dimension, comme renseignement ; c'est la clé que le fait porte. Elle a la même forme pour toutes les dimensions, quelle que soit celle du numéro, et elle ne dépend plus de la façon dont un système de l'entreprise numérote.

La clé se fabrique par `md5`. `md5(texte)` rend l'empreinte d'un texte : une chaîne de 32 caractères, toujours la même pour le même texte. Le numéro du produit est un entier, et `produit_id::varchar` en fait d'abord du texte. Comme l'empreinte ne dépend que du numéro, le fait la recalculera lui-même, sans aller la chercher dans la dimension. En entreprise, on l'écrit souvent par la macro `generate_surrogate_key` du paquet `dbt_utils`, qui produit ce même `md5`.

Une dimension reprend sa vue du `STAGING`, lui ajoute sa clé et ne garde que ce par quoi on regarde les ventes, ce que vous avez décidé au cours 02 : le nom, la catégorie et le format du produit ; le secteur postal et la date d'inscription du membre.

4.1 Écrire la dimension des produits.

```
notepad 02_ae\models\marts\dim_produit.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/models/marts/dim_produit.sql : un produit du catalogue par ligne
select
    -- 1. la clé de substitution, tirée du numéro du produit
    md5(produit_id::____) as cle_produit,
    -- 2. le numéro de la caisse, puis ce par quoi on regarde les ventes
    produit_id,
    -- puis, à écrire : nom, categorie et format
from {{ ref('____') }}
```

Enregistrer, puis fermer le Bloc-notes.

© 2026 iAlexMG. Tous droits réservés.

4.2 Écrire la dimension des membres. Le numéro de la carte est déjà du texte. La date d'inscription se tire de l'horodatage `inscrit_le`, sous le nom `date_inscription`.

```
notepad 02_ae\models\marts\dim_membre.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/models/marts/dim_membre.sql : un membre du programme de fidélité par ligne
select
    -- 1. la clé de substitution, tirée du numéro de la carte
    ____ as cle_membre,
    -- 2. le numéro de la carte, puis ce par quoi on regarde les ventes
    membre_id,
    -- puis, à écrire : secteur_postal, et inscrit_le en date, sous le nom date_inscription
from {{ ref('stg_membres') }}
```

Enregistrer, puis fermer le Bloc-notes.

4.3 Bâtir les deux dimensions. `--select` en prend plusieurs, séparées par une espace.

```
dbt run --select dim_produit dim_membre
REM Attendu : 2 of 2 OK created sql table model, main.dim_membre et main.dim_produit, puis PASS=2.
```

Étape 5 — Le fait, `fct_ventes` · MARTS

Le grain est fixé depuis le cours 02 : une ligne d'un ticket payé. `int_ventes` a fait la jointure et le filtre ; le fait ne garde que ses trois familles de colonnes, et se relit d'un coup d'œil.

La famille	**Dans `fct_ventes`**
Les clés vers les dimensions	`cle_date`, `cle_produit`, `cle_membre`
La dimension dégénérée, gardée dans le fait	`ticket_id`, `ligne`
Les mesures	`quantite`, `montant_brut`, `rabais`, `montant`

Les clés se recalculent comme dans les dimensions : le même `md5` du même numéro donne la même chaîne. La date de la vente sert de clé de date. Un ticket sans carte a un `membre_id` vide, et le `md5` d'une valeur vide reste vide : la clé du membre reste vide, comme sur l'étoile du cours 02. Le montant brut est la quantité fois le prix unitaire, avant le rabais.

5.1 Écrire le fait.

```
notepad 02_ae\models\marts\fct_ventes.sql
```


© 2026 iAlexMG. Tous droits réservés.

Accepter la création, puis écrire :

```
-- 02_ae/models/marts/fct_ventes.sql : le fait des ventes, une ligne par ligne d'un ticket payé
select
    -- 1. une clé vers chaque dimension, calculée comme dans la dimension
    date_vente as cle_date,
    md5(produit_id::varchar) as cle_produit,
    ____ as cle_membre,
    -- 2. le ticket et sa ligne, gardés dans le fait
    -- 3. les mesures
    quantite,
    ____ as montant_brut,
    -- puis, à écrire : rabais et montant
from {{ ref('int_ventes') }}
```

Enregistrer, puis fermer le Bloc-notes.

5.2 Bâtir tout le projet.

```
dbt run
REM Attendu : 9 of 9, cinq vues et quatre tables, puis Done. PASS=9 WARN=0 ERROR=0.
```

Les modèles sortent dans l'ordre que `ref()` dessine : `dim_date`, qui ne lit rien, peut partir le premier, mais `fct_ventes` attend `int_ventes`, qui attend `stg_lignes` et `stg_tickets`.

Si dbt s'arrête sur	**La cause, et le geste**
`Model 'model.brulerie.int_ventes' … depends on a node named 'stg_ligne' which was not found`	le nom dans `ref()` ne correspond à aucun modèle : le corriger, sans le `.sql`
`Binder Error: Referenced column "…" not found`	une colonne mal nommée ; le message donne le modèle, et `02_ae\target\compiled\…` le SQL envoyé
`IO Error: Cannot open file "…\brulerie.duckdb"`	le notebook tient l'entrepôt : quitter DuckDB dans la seconde fenêtre, puis relancer dbt

Étape 6 — Lire l'étoile, puis la réconcilier · MARTS

Un fait se bâtit à partir de plusieurs décisions : quel état courant, quelle jointure, quel filtre. Chacune peut être fausse sans rien casser, et un total faux se lit comme un total juste. Deux contrôles décident donc s'il se livre : son grain, puis un témoin indépendant.

6.1 Rouvrir le notebook, dans la seconde fenêtre.

© 2026 iAlexMG. Tous droits réservés.

```
duckdb -ui
```

Ouvrir le notebook « AE Cours 04 » et relancer sa première cellule, celle qui attache l'entrepôt.

6.2 Compter les jours de la dimension des dates, puis lire sa première semaine, chaque requête dans sa cellule.

```
select count(*) as jours, min(date_jour) as premier, max(date_jour) as dernier
from brulerie.dim_date;
-- Attendu : 30 jours, du 2025-06-01 au 2025-06-30.

select date_jour, nom_jour, est_fin_de_semaine
from brulerie.dim_date
order by date_jour
limit 7;
-- Attendu : le 2025-06-01, un dimanche, en fin de semaine (true), puis du lundi 2 au samedi 7.
```

6.3 Compter les lignes et les clés de la dimension des produits. Une clé de substitution est unique quand il y a autant de clés que de lignes.

```
select count(*) as lignes, count(distinct cle_produit) as cles
from brulerie.dim_produit;
-- Attendu : 32 lignes, 32 clés.
```

La même requête sur `dim_membre` et `cle_membre` rend 994 lignes et 994 clés.

6.4 Recalculer une clé à la main.

```
select produit_id, cle_produit, md5('101') as a_la_main
from brulerie.dim_produit
where produit_id = 101;
-- Attendu : la même chaîne deux fois, 38b3eff8baf56627478ec76a704e9b52.
```

La clé n'a rien de magique : le jour où un chiffre vous surprendra, vous pourrez la recalculer.

6.5 Contrôler le grain : aucun couple ticket et ligne ne doit paraître deux fois dans le fait. `having` filtre les groupes, comme `where` filtre les lignes.

```
-- 1. les couples (ticket, ligne) que le fait porte plus d'une fois
select ticket_id, ligne, count(*) as fois
from brulerie.fct_ventes
group by ticket_id, ligne
having count(*) > ____;
-- Attendu : aucune ligne.
```

Une ligne ici voudrait dire qu'une jointure a dupliqué des lignes : plus aucun total ne tiendrait.

6.6 Additionner le fait.

© 2026 iAlexMG. Tous droits réservés.

```
select count(*) as lignes, sum(montant_brut) as brut, sum(rabais) as rabais,
       sum(montant) as montant
from brulerie.fct_ventes;
-- Attendu : 2509 lignes ; 16836.75 brut, 118.75 de rabais, 16718.00 de montant.
```

Les 118,75 $ de rabais sont les boissons offertes aux membres contre leurs points.

6.7 Interroger le témoin. Le fait part des lignes ; la caisse, elle, a calculé le sous-total de chaque ticket de son côté. Deux chemins qui ne se croisent pas, et le même chiffre au bout : c'est ce qui fait de la caisse un témoin.

```
-- 1. le témoin : la somme des sous-totaux des tickets payés, tels que la caisse les a imprimés
select sum(sous_total) as caisse
from brulerie.stg_tickets
where statut = '____';
-- Attendu : 16718.00, au cent près le montant du fait.
```

L'écart vaut 0,00 $, et c'est le seul chiffre du cours qui n'en admet aucun. Sans le filtre de `int_ventes`, le fait aurait gardé les 23 lignes des 15 tickets annulés, pour 16 854,00 $: un total parfaitement plausible, que seul le témoin aurait trahi.

6.8 Répondre à la première question du mandat : ce qui s'est vendu en juin, par catégorie. Le fait rejoint sa dimension par la clé.

```
-- 1. le montant vendu par catégorie : le fait joint à la dimension des produits
select p.categorie, sum(f.montant) as montant
from brulerie.fct_ventes as f
inner join brulerie.dim_produit as p on p.cle_produit = f.____
group by p.categorie
order by p.categorie;
-- Attendu : 6 catégories, de ACCESSOIRE 627.00 à VIENNOISERIE 2407.25 ; les boissons chaudes
-- en tête, 5638.00. Leur somme fait 16718.00.
```

6.9 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 7 — Versionner · MARTS

7.1 Valider le fichier du projet et les nouveaux modèles, dans la première fenêtre.

```
git -C 02_ae add dbt_project.yml models
git -C 02_ae commit -m "Cours 04 : l'étoile des ventes"
REM Attendu : 6 files changed.
```


© 2026 iAlexMG. Tous droits réservés.

```
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Une règle métier s'écrit une fois, dans l'`INTERMEDIATE` : une vente est une ligne d'un ticket payé ; les annulés sortent, le remboursement reste.
- Chaque dimension a sa clé de substitution, que le fait recalcule sans jointure ; la date est sa propre clé.
- Un fait se contrôle avant de se livrer : son grain, puis un témoin indépendant, au cent près.

© 2026 iAlexMG. Tous droits réservés.